IRIS FLOWER CLASSIFICATION

A Machine Learning Classification Project

PROJECT REPORT

Prepared by: Niresh

Platform: Google Colab | Language: Python

Libraries: pandas, NumPy, Matplotlib, Seaborn, scikit-learn

Project overview

This project trains and compares three supervised machine-learning models to predict an Iris flower species from four measurements. It includes data inspection, preprocessing, model comparison, evaluation visualizations, and a sample prediction.

1. Abstract

The Iris Flower Classification project demonstrates an end-to-end machine-learning workflow. It uses the classic dataset of 150 observations across Iris setosa, Iris versicolor, and Iris virginica. Four numerical features—sepal length, sepal width, petal length, and petal width—are used as inputs. Logistic Regression, K-Nearest Neighbors (KNN), and Random Forest are evaluated on a stratified 80:20 train-test split. Accuracy, macro precision, macro recall, macro F1-score, and a confusion matrix are used to compare performance. Preprocessing is placed inside pipelines to reduce data leakage.

2. Problem Statement

Given the measurements of an Iris flower, predict its species among three known classes. This is a supervised multi-class classification problem.

3. Objectives

• Inspect a labelled dataset.
• Handle missing or invalid numeric values safely.
• Split the data while preserving class proportions.
• Train three classification algorithms.
• Compare multiple evaluation metrics.
• Visualize accuracy and class-level errors.
• Predict a species for a new measurement sample.

4. Dataset Description

The Iris dataset is bundled with scikit-learn, so no separate Kaggle download is required. It contains 150 rows, four numeric input features, and three target classes. It is a small educational benchmark; results should not be assumed to generalize to all real-world flower data.

Feature	Description
Sepal length (cm)	Length of sepal in centimetres
Sepal width (cm)	Width of sepal in centimetres
Petal length (cm)	Length of petal in centimetres
Petal width (cm)	Width of petal in centimetres
Target	Setosa, versicolor, or virginica

5. Tools and Technologies

Python is the implementation language. Google Colab provides the notebook runtime. pandas and NumPy support data handling; Matplotlib and Seaborn create visualizations; scikit-learn provides the dataset, preprocessing, models, and metrics.

6. Methodology

The data is split into 80% training and 20% testing with stratification and random_state=42. Median imputation is included as a safe preprocessing step. StandardScaler is applied to Logistic Regression and KNN because feature scale matters to these algorithms. Random Forest uses imputation without scaling. Pipelines fit transformations on training data only.

7. Algorithms

Logistic Regression: a linear classifier that estimates class probabilities.
K-Nearest Neighbors: predicts using nearby training examples; this project uses k=5.
Random Forest: combines multiple decision trees; this project uses 100 trees.

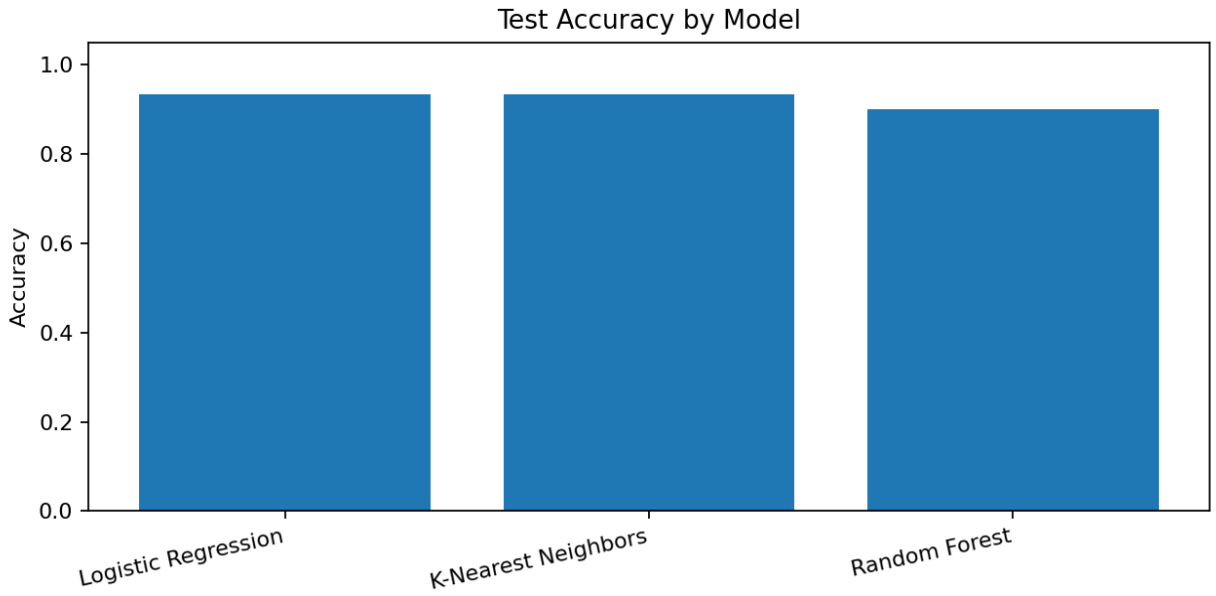
8. Evaluation Metrics

Accuracy is the proportion of test samples predicted correctly. Precision measures the correctness of class predictions; recall measures how many examples of a class are found; F1-score balances precision and recall. Macro averaging gives each class equal weight. The confusion matrix compares actual and predicted classes.

9. Experimental Results

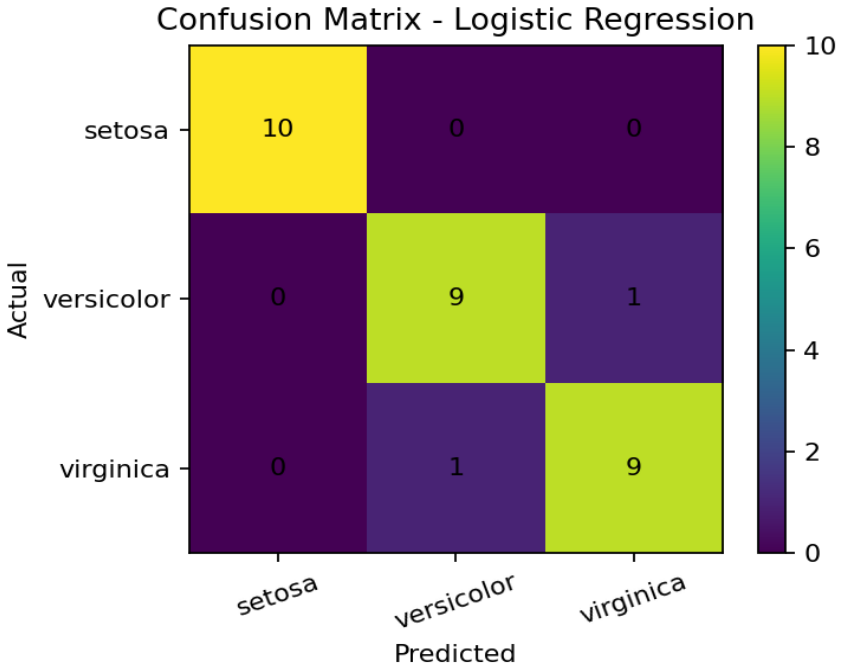
The following metrics were computed using a stratified 80:20 split and random_state=42. Results are specific to this run and can change if the split or settings change.

Model	Accuracy	Macro P	Macro R	Macro F1
Logistic Regression	0.933	0.933	0.933	0.933
K-Nearest Neighbors	0.933	0.944	0.933	0.933
Random Forest	0.900	0.902	0.900	0.900



The highest test accuracy in this run was achieved by **Logistic Regression** (93.33%). Consider all metrics and the confusion matrix, not accuracy alone.

10. Confusion Matrix



For the selected model, 28 of 30 test examples were classified correctly and 2 were misclassified. Rows represent actual classes; columns represent predicted classes.

11. Sample Prediction

The notebook predicts a sample with sepal length 5.1 cm, sepal width 3.5 cm, petal length 1.4 cm, and petal width 0.2 cm. The predicted species and probability estimates are printed when the notebook runs.

12. Conclusion

The project demonstrates a complete supervised classification workflow and compares three algorithms. In this reproducible run, Logistic Regression achieved the highest test accuracy among the evaluated models. The confusion matrix makes class-level errors visible. Because Iris is a small educational dataset, the result is not a guarantee of performance on unrelated real-world data.

13. Limitations and Future Scope

The dataset is small, hyperparameters are fixed, and only one train-test split is evaluated. Future work could add cross-validation, GridSearchCV, a Streamlit interface, model persistence, and testing on independent data.

14. How to Run in Google Colab

1. Create a notebook named Project2_Iris_Flower_Classification.
2. Paste the companion Python code into a cell or upload the .py file.
3. Run all cells from top to bottom.
4. Check the metrics, accuracy chart, confusion matrix, and sample prediction.
5. Save the notebook to Drive and use the actual displayed results in your submission.

15. Viva Questions and Answers

Q: What is classification? A: A supervised learning task that assigns an input to a category.
Q: Why split the dataset? A: To estimate performance on unseen samples.
Q: Why scale KNN features? A: Distance-based methods can be dominated by features with larger scales.
Q: What does a confusion matrix show? A: Counts of actual and predicted classes.
Q: Does high accuracy guarantee real-world performance? A: No; independent testing and cross-validation provide

stronger evidence.

Appendix A — Complete Python Source Code

The companion file Project2_Iris_Flower_Classification.py contains the full runnable implementation.

```
# Iris Flower Classification - Google Colab
import warnings
warnings.filterwarnings("ignore")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report, confusion_matrix

# 1. Load built-in dataset (no CSV download required)
iris = load_iris(as_frame=True)
X = iris.data.copy()
y = iris.target.copy()
feature_names = list(X.columns)
target_names = iris.target_names
print("Dataset shape:", X.shape)
print("Classes:", list(target_names))
display(X.head())
display(X.describe())
print("Missing values:\n", X.isnull().sum())

# 2. Clean and split data. Stratification preserves class proportions.
X = X.replace([np.inf, -np.inf], np.nan)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
print("Training rows:", len(X_train), "| Test rows:", len(X_test))

# 3. Pipelines keep imputation/scaling fitted only on training data.
models = {
    "Logistic Regression": make_pipeline(
        SimpleImputer(strategy="median"), StandardScaler(),
        LogisticRegression(max_iter=1000, random_state=42)),
    "K-Nearest Neighbors": make_pipeline(
        SimpleImputer(strategy="median"), StandardScaler(),
        KNeighborsClassifier(n_neighbors=5)),
    "Random Forest": make_pipeline(
        SimpleImputer(strategy="median"),
        RandomForestClassifier(n_estimators=100, random_state=42))
}

# 4. Train and compare models
```

Appendix A — Source Code (continued)

```
rows, trained, preds = [], {}, {}
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    trained[name], preds[name] = model, pred
    rows.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, pred),
        "Macro Precision": precision_score(y_test, pred, average="macro", zero_division=0),
        "Macro Recall": recall_score(y_test, pred, average="macro", zero_division=0),
        "Macro F1": f1_score(y_test, pred, average="macro", zero_division=0)
    })
results = pd.DataFrame(rows).sort_values("Accuracy", ascending=False)
display(results.round(3))

# 5. Accuracy chart
plt.figure(figsize=(8,4))
plt.bar(results["Model"], results["Accuracy"])
plt.title("Test Accuracy by Model")
plt.ylabel("Accuracy")
plt.ylim(0, 1.05)
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()

# 6. Best model and detailed evaluation
best_name = results.iloc[0]["Model"]
best_model = trained[best_name]
best_pred = preds[best_name]
print("Best model:", best_name)
print(classification_report(
    y_test, best_pred, labels=np.arange(len(target_names)),
    target_names=target_names, zero_division=0
))

# 7. Confusion matrix
cm = confusion_matrix(y_test, best_pred, labels=np.arange(len(target_names)))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=target_names, yticklabels=target_names)
plt.title("Confusion Matrix - " + best_name)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()

# 8. Predict one new sample (measurements are in centimetres)
sample = pd.DataFrame([[5.1, 3.5, 1.4, 0.2]], columns=feature_names)
predicted_class = best_model.predict(sample)[0]
print("Sample measurements:")
display(sample)
```

Appendix A — Source Code (continued)

```
print("Predicted species:", target_names[predicted_class])
if hasattr(best_model, "predict_proba"):
    for flower, probability in zip(target_names, best_model.predict_proba(sample)[0]):
        print(f"{flower}: {probability:.3f}")

# 9. Automatically print a conclusion based on measured results
print("\nCONCLUSION")
print(f"Best test accuracy: {best_name} ({results.iloc[0]['Accuracy']*100:.2f}%)")
print(f"Correct predictions: {int(np.trace(cm))}/{int(cm.sum())}")
print("These results apply to this fixed test split; validate further before real-world use.")
```